
Self-Adjoint Flow Policy Optimization

Anonymous Author(s)

Affiliation

Address

email

Abstract

Generative policies offer a promising alternative to Gaussian policies in reinforcement learning (RL) due to their ability to represent expressive and multimodal action distributions. However, integrating such policies into likelihood-based policy optimization remains challenging, since accurate and differential action likelihoods are intractable for standard generative policies. In this paper, we propose *Self-Adjoint Flow Policy Optimization* (SAFLOW), a likelihood-tractable generative policy with self-adjoint structure. SAFLOW parameterizes the policy as an invertible flow in a doubled action space and uses a Verlet-style self-adjoint composition for generation. This structure makes the inverse map obtainable by stepsize reversal, aligning forward sampling and backward likelihood evaluation under the same time-symmetric numerical rule. It also provides a second-order approximation accuracy to the learned continuous flow, improving the numerical consistency of likelihood and policy-ratio computation. We instantiate SAFLOW within an on-policy optimization framework and evaluate it on eight IsaacLab continuous-control tasks. SAFLOW consistently outperforms Gaussian-policy baselines and recent generative-policy methods, highlighting the value of self-adjoint numerical structure for likelihood-based generative policy optimization.

1 Introduction

Expressive stochastic policies are crucial for reinforcement learning (RL) in high-dimensional continuous-control problems. While standard Gaussian policies remain widely used because they provide tractable likelihoods and admit simple reparameterized sampling [1, 2, 3], their unimodal form can be restrictive in settings involving multimodal exploration, contact-rich manipulation, locomotion, or coordinated whole-body control [4, 5]. Generative policies, including diffusion and flow-based policies, offer a more flexible alternative by representing complex action distributions through learned state-conditioned generative dynamics [6, 7, 8].

Despite this expressiveness, deploying generative policies in likelihood-based policy optimization remains challenging because they often lack tractable and numerically reliable likelihood evaluation [9]. In particular, the finite-step discretized generation process is typically not an explicitly invertible map with a tractable density. As a result, the forward sampling procedure and the backward likelihood computation may become inconsistent, leading to unstable likelihood ratios and unreliable policy-gradient updates.

Recent work has explored several ways to make expressive generative policies compatible with policy-gradient optimization. FPO incorporates flow matching [10] into a PPO-style policy-gradient framework by deriving an advantage-weighted surrogate ratio from the conditional flow-matching loss, thereby avoiding explicit likelihood computation while retaining the expressiveness of flow-based policy parameterizations [9]. GenPO instead directly addresses likelihood tractability by introducing a doubled action space and an alternating update rule, making the discretized generative process explicitly invertible and enabling tractable change-of-variables likelihood evaluation [11].

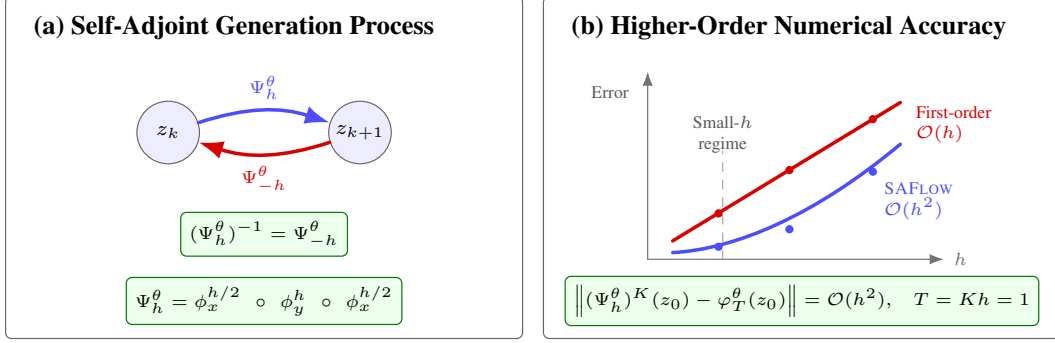


Figure 1: Key properties of the proposed SAFLOW. **Left:** the numerical update is exactly invertible by reversing the stepsize, so forward sampling and backward likelihood evaluation are implemented by the same rule in opposite time directions. **Right:** SAFLOW achieves second-order accuracy, leading to significantly smaller discretization error in the practically relevant small-stepsize regime.

39 However, this alternating update is first-order and asymmetric: although an inverse map can be
 40 computed explicitly, it is not, in general, obtained by applying the same update rule with the opposite
 41 stepsize. Consequently, forward sampling and backward likelihood evaluation are not governed by a
 42 single time-symmetric numerical scheme.

43 We argue that this numerical structure matters for likelihood-based generative policy optimization [12].
 44 A generative policy is not merely a sampler; it also defines the likelihoods used to compute policy
 45 gradients, entropy estimates, KL penalties, and PPO-style likelihood ratios [1]. Exact invertibility
 46 ensures that a density can be evaluated, but it does not ensure that the forward sampler and backward
 47 likelihood evaluator are numerically aligned. If these two computations are induced by asymmetric
 48 discretizations, the resulting likelihood ratios may inherit discretization artifacts. We therefore
 49 propose to strengthen invertibility into self-adjoint numerical consistency: the inverse should be
 50 obtained by the same update rule run backward in time, and the discrete generative map should
 51 accurately approximate the learned continuous flow.

52 In this work, we propose *Self-Adjoint Flow Policy Optimization* (SAFLOW), a generative policy
 53 optimization framework built around a self-adjoint flow policy. SAFLOW operates in a doubled
 54 action space and replaces first-order alternating updates with a Verlet-style self-adjoint composition.
 55 As shown in Figure 1, this design preserves exact numerical inversion and makes forward and
 56 backward computations time-symmetric: $(\Psi_h^\theta)^{-1} = \Psi_{-h}^\theta$. Consequently, sampling and likelihood
 57 evaluation are implemented by the same update rule in opposite time directions. Moreover, the
 58 symmetric composition provides a second-order approximation to the learned continuous flow:
 59 $\|(\Psi_h^\theta)^K(z_0) - \varphi_T^\theta(z_0)\| = \mathcal{O}(h^2)$, $T = Kh = 1$, where φ_T^θ denotes the exact time- T flow
 60 induced by the learned continuous velocity field on the variable z .

61 Our contributions are as follows.

- 62 • We propose SAFLOW, a likelihood-tractable generative policy optimization framework
 63 that strengthens exact invertibility into self-adjoint numerical consistency. By operating
 64 in a doubled action space and using a Verlet-style kick–drift–kick composition, SAFLOW
 65 implements forward sampling and backward likelihood evaluation with the same update rule
 66 in opposite time directions.
- 67 • We prove that the proposed self-adjoint flow admits exact inversion by stepsize reversal and
 68 provides a second-order approximation to the learned continuous flow. We further show that
 69 this second-order consistency propagates to log-likelihoods, policy ratios, and PPO-style
 70 clipped surrogate objectives.
- 71 • We evaluate SAFLOW on eight IsaacLab continuous-control tasks spanning locomotion,
 72 manipulation, dexterous control, and aerial control. Across these diverse domains, SAFLOW
 73 consistently outperforms Gaussian-policy baselines and recent generative-policy methods,
 74 achieving higher final returns and more stable learning.

75 2 Preliminaries

76 **Likelihood-Based Policy Optimization in RL.** We consider a Markov decision process $\mathcal{M} =$
 77 $(\mathcal{S}, \mathcal{A}, P, r, \rho_0, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s'|s, a)$ is the transition kernel,
 78 $r(s, a)$ is the reward function, ρ_0 is the initial state distribution, and $\gamma \in [0, 1)$ is the discount fac-
 79 tor [13]. A stochastic policy $\pi_\theta(a|s)$ induces a trajectory distribution over $\tau = (s_0, a_0, s_1, a_1, \dots)$,
 80 and the standard RL objective is $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$. Many policy optimization algo-
 81 rithms rely on evaluating the action likelihood $\pi_\theta(a|s)$ or its logarithm for encouraging exploration
 82 or computing policy gradients. For example, policy-gradient methods with importance sampling use
 83 likelihood ratios of the form $r_\theta(s, a) = \frac{\pi_\theta(a|s)}{\mu(a|s)}$, where μ denotes the behavior policy that generated
 84 the data [1, 2]. In on-policy PPO-style optimization, the clipped surrogate objective is

$$\mathcal{L}_{\text{clip}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right], \quad (1)$$

85 where $\hat{A}_t$ is advantage estimate and $\epsilon > 0$ is the clipping threshold. Action log-likelihoods also appear
 86 in KL-based trust-region constraints and entropy-regularized objectives [2, 3]. Overall, accurate and
 87 differentiable likelihood evaluation is a fundamental requirement for a broad class of RL algorithms.

88 **Generative Policies for RL.** A generative policy samples an action by transforming a simple
 89 base random variable. Let $z_0 \sim p_0(z)$ be a base noise variable, usually a standard Gaussian. A
 90 state-conditioned generative policy defines a transformation $z_K = F_\theta(z_0; s)$, where $F_\theta(\cdot; s)$ may be
 91 implemented by a diffusion model [14] or flow matching velocity [4, 15]. The generated action is
 92 obtained as z_K . When $F_\theta(\cdot; s)$ is invertible, the density of the generated variable can be evaluated by
 93 the change-of-variables formula:

$$\log \pi_\theta(z_K|s) = \log p_0(z_0) - \log \left| \det \frac{\partial F_\theta(z_0; s)}{\partial z_0} \right|. \quad (2)$$

94 This formula is the basis of change of variables. However, standard diffusion/flow policies are not
 95 directly likelihood-tractable under finite-step discretization, because the sampling process is generally
 96 not written as an exactly invertible map with a tractable Jacobian [11]. This creates a gap between
 97 the expressiveness of diffusion/flow policies and the likelihood requirements of policy optimization.

98 3 Method

99 3.1 SAFLOW: Self-Adjoint Flow Policy

100 SAFLOW defines a state-conditioned invertible policy in an augmented action space. Let the
 101 environment action space be $\mathcal{A} \subseteq \mathbb{R}^d$. We adopt a doubled latent action $z = (x, y) \in \mathbb{R}^{2d}$, which
 102 provides sufficient degrees of freedom for invertible density modeling. Specifically, given a state
 103 s , SAFLOW samples a base variable $z_0 = (x_0, y_0) \sim p_0$, where p_0 is a standard Gaussian, and
 104 transform it with a K -step state-conditioned flow $z_K = F_\theta(z_0; s)$, $z_K = (x_K, y_K)$. The dummy
 105 action is $\tilde{a} = z_K$, and the action executed in the environment is $a = \mathcal{M}(z_K) = (x_K + y_K)/2$. The
 106 resulting stochastic policy is therefore induced by the invertible map F_θ in the augmented space.

107 The core design of SAFLOW is the construction of F_θ . Let $z_k = (x_k, y_k)$ denote the augmented
 108 variable at step k , and let $v_\theta(\cdot, t; s)$ be a state-conditioned velocity field. Rather than using a one-sided
 109 update, SAFLOW advances the two dummy-action components through a symmetric Verlet-style
 110 kick-drift-kick technique:

$$\begin{aligned} \phi_x^{h/2} : \quad x_{k+\frac{1}{2}} &= x_k + \frac{h}{2} v_\theta(y_k, t_k; s), \\ \phi_y^h : \quad y_{k+1} &= y_k + h v_\theta \left(x_{k+\frac{1}{2}}, t_k + \frac{h}{2}; s \right), \\ \phi_x^{h/2} : \quad x_{k+1} &= x_{k+\frac{1}{2}} + \frac{h}{2} v_\theta(y_{k+1}, t_{k+1}; s). \end{aligned} \quad (3)$$

111 We denote the resulting one-step map by $z_{k+1} = \Psi_h^\theta(z_k; s)$. Equivalently, the step can be written as
 112 the symmetric composition $\Psi_h^\theta = \phi_x^{h/2} \circ \phi_y^h \circ \phi_x^{h/2}$, where ϕ_x and ϕ_y denote partial shear updates of

the two dummy-action components. The full generative policy flow is the composition of K such self-adjoint steps, $F_\theta(\cdot; s) = (\Psi_h^\theta)^K(\cdot; s)$.

This construction has two immediate benefits. First, each partial update is explicitly invertible, so the whole generative policy admits tractable backward evaluation. Second, because the composition is symmetric, the inverse step is obtained by applying the same rule with the opposite stepsize. Thus, sampling and likelihood evaluation are aligned by the same numerical scheme in opposite time directions. In this sense, SAFLOW upgrades invertible generative policies from merely having an inverse map to having a time-symmetric self-adjoint flow.

We next describe likelihood evaluation. Given a dummy action z_K , we recover its base variable by self-adjoint inversion: $z_0 = F_\theta^{-1}(z_K; s) = (\Psi_{-h}^\theta)^K(z_K; s)$. By the change-of-variables formula,

$$\log \pi_\theta(z_K | s) = \log p_0(z_0) - \sum_{k=0}^{K-1} \log |\det D\Psi_h^\theta(z_k; s)|. \quad (4)$$

The likelihood depends on θ through the inverse map F_θ^{-1} , which determines the base point assigned to the observed dummy action. This enables tractable likelihood ratios for on-policy optimization while preserving an expressive state-conditioned generative policy.

3.2 SAFLOW Policy Optimization in the Augmented MDP

SAFLOW can be interpreted as performing policy optimization in an augmented MDP $\widetilde{\mathcal{M}} = (\mathcal{S}, \widetilde{\mathcal{A}}, \widetilde{P}, \widetilde{r}, \rho_0, \gamma)$, where $\widetilde{P}(s'|s, \widetilde{a}) = P(s'|s, M(\widetilde{a}))$, $\widetilde{r}(s, \widetilde{a}) = r(s, M(\widetilde{a}))$. This formulation makes the likelihood ratio used by PPO well-defined in the augmented action space: $r_\theta(s, \widetilde{a}) = \widetilde{\pi}_\theta(\widetilde{a}|s)/\widetilde{\pi}_{\theta_{\text{old}}}(\widetilde{a}|s)$. Thus, SAFLOW does not require the marginal density of the projected action $a = M(\widetilde{a})$. Instead, it performs standard likelihood-ratio policy optimization in the augmented MDP, whose rewards and transitions depend on $\widetilde{a}$ only through $M(\widetilde{a})$.

Given rollout data generated by the old policy $\widetilde{\pi}_{\theta_{\text{old}}}$, SAFLOW optimizes the clipped PPO surrogate

$$L_{\text{clip}}(\theta) = \mathbb{E} \left[\min \left(r_\theta(s_t, \widetilde{a}_t) \widehat{A}_t, \text{clip}(r_\theta(s_t, \widetilde{a}_t), 1 - \epsilon, 1 + \epsilon) \widehat{A}_t \right) \right], \quad (5)$$

where $\widehat{A}_t$ is estimated from the actual rewards obtained after executing $M(\widetilde{a}_t)$.

The doubled action space introduces redundant degrees of freedom because many dummy actions induce the same environment action. To regularize this projection null space, SAFLOW adds a compression penalty

$$L_{\text{comp}}(\theta) = \mathbb{E}_{\widetilde{a}=(x,y) \sim \widetilde{\pi}_\theta(\cdot|s)} [\|x - y\|_2^2]. \quad (6)$$

The final actor objective is

$$L_{\text{policy}}(\theta) = L_{\text{clip}}(\theta) - \nu L_{\text{comp}}(\theta), \quad (7)$$

where $\nu > 0$ controls the compression strength. The compression term is not required for the correctness of the augmented policy-gradient objective; rather, it selects compact representatives among equivalent dummy-action lifts. The complete pseudocode is provided in Algorithm 1.

3.3 Theoretical Analysis

We analyze SAFLOW from two complementary perspectives. First, we show that optimizing dummy-action likelihoods is principled because the doubled action space defines an augmented MDP that preserves the original optimal control problem. Second, we show that the self-adjoint numerical structure yields exact inversion and second-order consistency of likelihood ratios and PPO surrogates.

3.3.1 Principled Optimization in the Augmented Action Space

Theorem 1 (Optimality Preservation under Augmented Actions). *Let $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \rho_0, \gamma)$ be the original MDP. Define the augmented action space $\widetilde{\mathcal{A}} = \mathcal{A} \times \mathcal{A}$ and the projection $M(x, y) = (x + y)/2$. Let $\widetilde{\mathcal{M}} = (\mathcal{S}, \widetilde{\mathcal{A}}, \widetilde{P}, \widetilde{r}, \rho_0, \gamma)$ be the augmented MDP. Then optimizing over augmented-action policies does not change the optimal control value: $\sup_{\widetilde{\pi}} J_{\widetilde{\mathcal{M}}}(\widetilde{\pi}) = \sup_{\pi} J_{\mathcal{M}}(\pi)$.*

Algorithm 1 SAFLOW Policy Optimization

Require: policy $\tilde{\pi}_\theta$, critic V_ψ , flow steps K , step size $h = 1/K$, compression coefficient ν

```

1: for each training iteration do
2:   Set  $\theta_{\text{old}} \leftarrow \theta$  and collect buffer  $\mathcal{B}$ 
3:   for each environment step do
4:     Sample  $z_0 = (x_0, y_0) \sim p_0$  and generate  $\tilde{a} = (x, y) = (\Psi_h^{\theta_{\text{old}}})^K(z_0|s)$ 
5:     Compute and store old log-prob  $\ell_{\text{old}} = \log \tilde{\pi}_{\theta_{\text{old}}}(\tilde{a}|s)$ 
6:     Execute  $a = M(\tilde{a}) = (x + y)/2$  and store  $(s, \tilde{a}, r, s', \ell_{\text{old}})$ 
7:   end for
8:   for each minibatch do
9:     Evaluate  $\log \tilde{\pi}_\theta(\tilde{a}|s)$  by (4)
10:    Update policy  $\tilde{\pi}_\theta$  by (7)
11:    Update critic  $V_\psi$  by standard TD error
12:   end for
13: end for

```

152 *Proof sketch.* For any augmented policy $\tilde{\pi}$, the projected policy $M_\# \tilde{\pi}$ induces the same distribution
 153 over environment states and executed actions, since both $\tilde{P}$ and $\tilde{r}$ depend on $\tilde{a}$ only through $M(\tilde{a})$.
 154 Therefore $J_{\tilde{\mathcal{M}}}(\tilde{\pi}) = J_{\mathcal{M}}(M_\# \tilde{\pi})$. Conversely, any original policy π can be lifted to the diagonal
 155 policy $\tilde{\pi}^\Delta$ that samples $a \sim \pi(\cdot|s)$ and sets $(x, y) = (a, a)$. This lift satisfies $J_{\tilde{\mathcal{M}}}(\tilde{\pi}^\Delta) = J_{\mathcal{M}}(\pi)$.
 156 Combining both directions proves the result. See Appendix A.1 for more details. $\square$

157 **Theorem 2** (Policy Gradient in the Augmented Action Space). *Assume that $\tilde{\pi}_\theta(\tilde{a}|s)$ is differentiable*
 158 *in θ and has support independent of θ . Then the policy gradient in the augmented MDP satisfies*

$$\nabla_\theta J_{\tilde{\mathcal{M}}}(\theta) = \mathbb{E}_{(s, \tilde{a}) \sim d^{\tilde{\pi}_\theta}} \left[\nabla_\theta \log \tilde{\pi}_\theta(\tilde{a}|s) A^{\tilde{\pi}_\theta}(s, \tilde{a}) \right],$$

159 *where $d^{\tilde{\pi}_\theta}$ denotes the discounted state–action occupancy measure in $\tilde{\mathcal{M}}$. Consequently, for samples*
 160 *collected by $\tilde{\pi}_{\theta_{\text{old}}}$, the corresponding one-step importance ratio is $r_\theta(s, \tilde{a}) = \tilde{\pi}_\theta(\tilde{a}|s)/\tilde{\pi}_{\theta_{\text{old}}}(\tilde{a}|s)$,*
 161 *which is the likelihood ratio used in PPO-style optimization over the augmented action space.*

162 *Proof sketch.* The result follows directly from the standard policy-gradient theorem applied to the
 163 augmented MDP $\tilde{\mathcal{M}}$. In this MDP, the policy acts on the dummy action $\tilde{a}$, and the corresponding
 164 policy density is $\tilde{\pi}_\theta(\tilde{a}|s)$. Therefore, the score term in the policy gradient is $\nabla_\theta \log \tilde{\pi}_\theta(\tilde{a}|s)$, yielding
 165 the stated gradient expression. See Appendix A.2 for the full derivation. $\square$

166 3.3.2 Numerical Advantages of Self-Adjoint Flow

167 **Theorem 3** (Exact Time-Symmetric Inversion of the Self-Adjoint Flow). *Let Ψ_h^θ denote one SAFLOW*
 168 *integration step with stepsize h , defined by the symmetric composition $\Psi_h^\theta = \phi_x^{h/2} \circ \phi_y^h \circ \phi_x^{h/2}$.*
 169 *Then the inverse of one step is obtained by applying the same update with the opposite stepsize:*
 170 *$(\Psi_h^\theta)^{-1} = \Psi_{-h}^\theta$. Consequently, for the K -step generative map $F_{\theta,h} = (\Psi_h^\theta)^K$, the inverse map used*
 171 *for likelihood evaluation is $F_{\theta,h}^{-1} = (\Psi_{-h}^\theta)^K$.*

172 *Proof sketch.* The result follows from the symmetric, self-adjoint structure of the update. Each split
 173 map is exactly inverted by negating its stepsize: $(\phi_x^{h/2})^{-1} = \phi_x^{-h/2}$, $(\phi_y^h)^{-1} = \phi_y^{-h}$. Using the
 174 inverse rule for compositions, we obtain $(\Psi_h^\theta)^{-1} = \Psi_{-h}^\theta$. Applying this one-step inverse repeatedly
 175 yields $F_{\theta,h}^{-1} = (\Psi_{-h}^\theta)^K$. See Appendix A.3 for more details. $\square$

176 **Theorem 4** (Second-order approximation in C^1). *Assume that the split vector fields are sufficiently*
 177 *smooth on a compact domain and that both the exact and numerical trajectories remain in this domain*
 178 *over the time horizon T . Then the SAFLOW discretization satisfies $\sup_{z_0} \|F_{\theta,h}(z_0) - \varphi_T^\theta(z_0)\| =$*
 179 *$\mathcal{O}(h^2)$. If, in addition, the vector fields have one higher order of smoothness, then the Jacobians of the*
 180 *numerical and exact flows satisfy the same second-order bound: $\sup_{z_0} \|DF_{\theta,h}(z_0) - D\varphi_T^\theta(z_0)\| =$*
 181 *$\mathcal{O}(h^2)$. Together, these two estimates imply, $\|F_{\theta,h} - \varphi_T^\theta\|_{C^1} = \mathcal{O}(h^2)$.*

182 *Proof sketch.* The SAFLOW step is a symmetric Strang splitting $\Psi_h^\theta = \phi_x^{h/2} \circ \phi_y^h \circ \phi_x^{h/2}$. By the
 183 Baker–Campbell–Hausdorff expansion [16], the leading asymmetric error terms cancel, giving local
 184 error $\mathcal{O}(h^3)$; over $K = T/h$ steps this yields global error $\mathcal{O}(h^2)$. Applying the same argument to the
 185 variational dynamics gives the Jacobian error $\mathcal{O}(h^2)$, hence the C^1 bound. See Appendix A.4 for
 186 more details. $\square$

187 **Theorem 5** (Second-order likelihood consistency). *Let φ_T^θ be the exact continuous flow correspond-*
 188 *ing to the split dynamics, and with $\|F_{\theta,h} - \varphi_T^\theta\|_{C^1} = \mathcal{O}(h^2)$. Consider that the base density p_0*
 189 *is positive and continuously differentiable on the relevant compact domain, and that the Jacobian*
 190 *determinant of φ_T^θ is bounded away from zero. Let $\ell_{\theta,h}(s, z)$ be the log-likelihood induced by the*
 191 *discretized flow. Let $\ell_{\theta,\text{cont}}(s, z)$ be the corresponding log-likelihood under the exact continuous flow.*
 192 *Then $|\ell_{\theta,h}(s, z) - \ell_{\theta,\text{cont}}(s, z)| = \mathcal{O}(h^2)$ uniformly holds on the generated compact domain.*

193 *Proof sketch.* The C^1 closeness of $F_{\theta,h}$ and φ_T^θ , together with nondegeneracy of the exact flow,
 194 implies C^1 closeness of their inverse maps. The base-density term is then second-order accurate
 195 by Lipschitz continuity of $\log p_0$, and the log-determinant term is second-order accurate because
 196 $A \mapsto \log |\det A|$ is locally Lipschitz on nonsingular matrices. Combining the two terms gives the
 197 result. See Appendix A.5 for more details. $\square$

198 **Corollary 6** (Second-order consistency of policy-ratio and PPO surrogate). *Let $r_{\theta,h}(s, z) =$*
 199 *$\exp(\ell_{\theta,h}(s, z) - \ell_{\theta_{\text{old}},h}(s, z))$ be the SAFLOW policy ratio, and let $r_{\theta,\text{cont}}(s, z)$ be the correspond-*
 200 *ing exact-flow ratio. With Theorem 5 for both θ and θ_{old} , $r_{\theta,h}(s, z) = r_{\theta,\text{cont}}(s, z) \exp(\mathcal{O}(h^2))$. If*
 201 *the exact ratio is bounded and $|\hat{A}_t| \leq A_{\text{max}}$, then the clipped PPO surrogate computed with the*
 202 *discretized likelihood satisfies $|L_{\text{clip},h}(\theta) - L_{\text{clip},\text{cont}}(\theta)| = \mathcal{O}(h^2)$.*

203 *Proof sketch.* The ratio result follows by subtracting the second-order log-likelihood errors for θ and
 204 θ_{old} and exponentiating. The PPO result follows because the clipped surrogate is Lipschitz in the
 205 policy ratio with constant at most $|\hat{A}_t|$ for each sample. See Appendix A.6 for more details. $\square$

206 Together, these results show that SAFLOW is principled at the RL formulation level and consistent at
 207 the numerical likelihood level: the doubled action space defines an optimality preserved augmented
 208 control problem and valid dummy-action likelihood ratios; the self-adjoint flow yields exact time-
 209 symmetric inversion structure, second-order approximation accuracy of learned continuous-time flow,
 210 likelihood, policy-ratio and PPO surrogates.

211 4 Experiments

212 **Benchmarks.** We assess the performance of our approach using a collection of benchmark environ-
 213 nments from IsaacLab [17]. Specifically, we evaluate on eight tasks: Ant, Humanoid, Franka Arm,
 214 Shadow Hand, Anymal-D, Unitree-Go2, Unitree-H1, and Quadcopter, detailed in Appendix B.1.

215 **Baselines.** We benchmark SAFLOW against representative policy optimization algorithms. The
 216 comparison includes widely used Gaussian-policy methods: PPO [1] and TRPO [2], together with
 217 generative-policy approaches: FPO [9], GenPO [11]. To improve the reliability of the reported
 218 results, each experiment is carried out with five different random seeds. Additional experimental
 219 configurations and implementation details are provided in the Appendix B.2 and B.3.

220 **Main Results.** Figure 2 shows the learning curves of all methods, where SAFLOW generally learns
 221 faster and reaches higher final returns than the baselines. As summarized in Table 1, SAFLOW
 222 achieves the best mean return across all eight benchmark tasks, demonstrating consistent improve-
 223 ments over Gaussian-policy, and generative-policy baselines. The gains are especially pronounced
 224 on challenging Humanoid, Unitree locomotion and Franka Arm, Shadow Hand manipulation tasks,
 225 where high-dimensional control and complex contact dynamics make optimization more difficult.
 226 These results suggest that SAFLOW improves both sample efficiency and training stability across
 227 diverse IsaacLab environments.

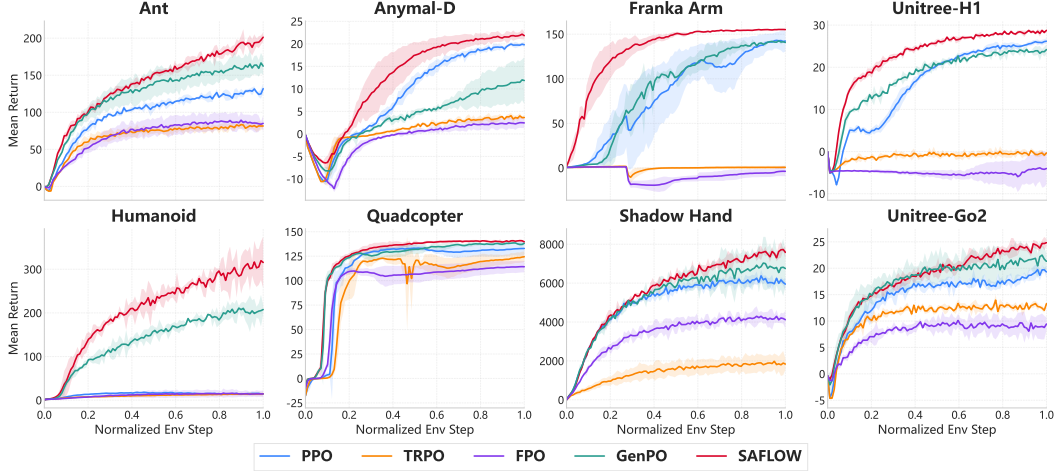


Figure 2: Learning curves across 8 IsaacLab tasks. Results are averaged over 5 runs. The x-axis denotes normalized env steps, and the y-axis shows average episodic return.

Table 1: Numerical results across 8 IsaacLab tasks.

Environment	PPO	TRPO	FPO	GenPO	SAFLOW
Ant	134.48 ± 1.72	83.98 ± 2.51	89.98 ± 11.54	168.92 ± 17.74	201.34 ± 5.63
Humanoid	18.91 ± 0.09	14.83 ± 2.70	16.59 ± 9.57	218.44 ± 8.23	329.60 ± 21.72
Franka Arm	143.53 ± 7.60	1.87 ± 0.98	1.22 ± 0.39	143.25 ± 1.41	155.57 ± 0.03
Anymal-D	20.13 ± 0.26	4.09 ± 0.11	2.70 ± 0.98	12.02 ± 4.61	22.15 ± 1.00
Unitree-Go2	20.44 ± 1.24	14.13 ± 0.70	10.56 ± 1.80	23.15 ± 2.57	25.51 ± 1.49
Unitree-H1	26.36 ± 0.42	0.65 ± 0.23	-0.02 ± 0.01	24.85 ± 0.45	29.21 ± 0.16
Quadcopter	133.39 ± 2.43	124.45 ± 4.61	114.44 ± 6.10	138.76 ± 1.08	140.93 ± 0.99
Shadow Hand	6430.09 ± 56.78	2035.10 ± 424.20	4427.83 ± 335.09	7188.21 ± 996.65	7984.18 ± 161.96

228 **Scaling with Parallel Environments.** We evaluate the scalability of SAFLOW on Ant by varying
 229 the number of GPU-parallel environments from 128 to 4096, with 2048 used by default in our
 230 main experiments. For clarity, Figure 3 reports the learning curves of three representative methods,
 231 PPO, GenPO, and SAFLOW; the complete set of curves is provided in Appendix C.5. SAFLOW
 232 outperforms these baselines when the number of parallel environments is relatively large (≥ 1024),
 233 and its performance further improves as more environments are used. These results suggest that
 234 SAFLOW effectively benefits from larger on-policy batches, where more diverse fresh rollouts reduce
 235 the variance of advantage and policy-ratio estimates within each update. This scaling behavior
 236 is enabled by the combination of an expressive generative policy, which exploits increased batch
 237 diversity, and a self-adjoint numerical structure, which provides stable and accurate likelihood
 238 evaluation for optimization.

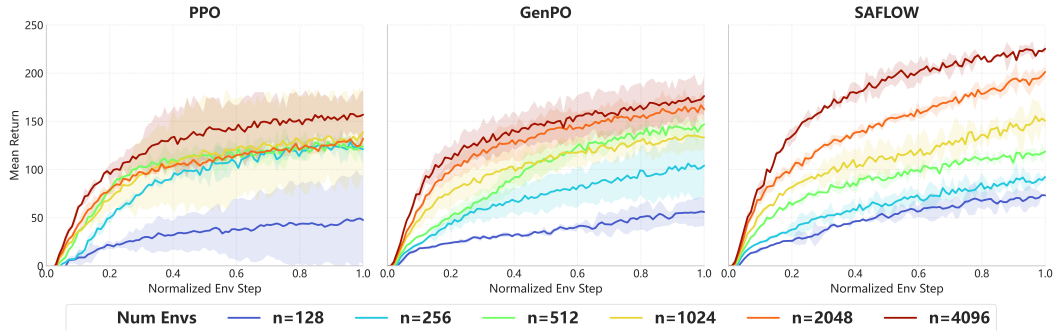


Figure 3: Learning curves of scaling experiment.

Ablation Study. To further examine which parts of SAFLOW are responsible for its performance gains, we perform a set of ablation experiments using the Ant task.

(1) Flow Steps. Figure 4(a) shows the influence of the number of flow steps K . Increasing K slightly improves performance, as a finer discretization provides a more accurate approximation to the learned continuous flow. However, the overall differences across different choices of K are modest, and the learning curves and final returns remain similar. This indicates that SAFLOW is relatively robust to this hyperparameter. We use $K = 5$ as the default setting, which provides a good balance between performance and computational cost.

(2) Compress Coefficient. Figure 4(b) studies the effect of the compression coefficient ν . The results show that a small compression weight is beneficial, with $\nu = 0.01$ achieving the best performance. Larger values overly restrict the policy and significantly reduce the return, while removing the compression term also performs worse. This confirms the importance of mild compression.

(3) Time Embedding Dimension. During training, rather than using the denoising time step t directly as a raw input feature, we encode this temporal variable with sinusoidal positional embeddings to better capture temporal relationships and ordering, as detailed in Appendix B.2. Figure 4(c) evaluates different time embedding dimensions e_t . Larger embedding dimensions generally improve performance, while too small a dimension limits the representation ability of the policy. Among the tested settings, $e_t = 32$ achieves the strongest final return, and is therefore used as default setting.

(4) Time Embedding Hidden Size. Figure 4(d) compares different hidden sizes h_{emb} in the time embedding network. The results suggest that increasing the hidden size improves the expressiveness of the embedding module and leads to better performance. We choose $h_{\text{emb}} = 256$ as the default configuration, since it obtains the best final return among the tested values.

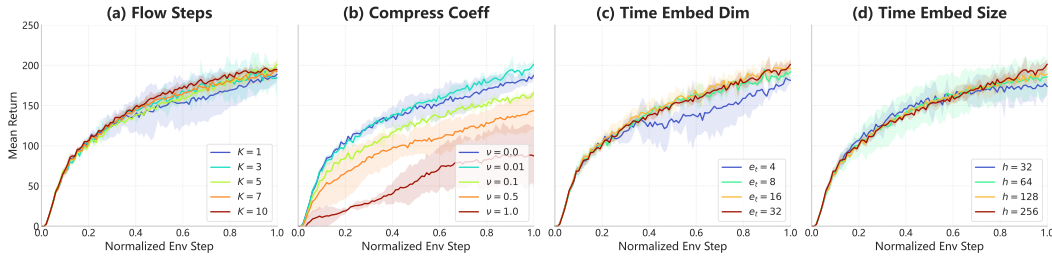


Figure 4: Learning curves of ablation study. Default setting: $K = 5$, $\nu = 0.01$, $e_t = 32$, $h_{\text{emb}} = 256$.

Additional Experiments. We conduct four additional experiments on Ant and Humanoid to further examine the efficiency, integration structure, and numerical behavior of SAFLOW.

(1) Training-time efficiency. Appendix C.1 reports wall-clock training time and performance-versus-time curves. Due to self-adjoint flow sampling and backward likelihood evaluation, SAFLOW requires approximately 32% and 23% more training time than GenPO on Ant and Humanoid, respectively. Despite this additional cost, its performance-versus-time curves remain competitive as shown in Figure 7, showing that the accuracy gains from the self-adjoint integration scheme are not solely a consequence of substantially increased computation.

(2) Controlled comparison of integration schemes. GenPO differs from SAFLOW not only in the numerical integration rule, but also in several auxiliary design choices, such as action mixing and customized entropy regularization. To isolate the effect of the integration scheme, Appendix C.2 provides a controlled comparison within our SAFLOW implementation, where we replace the self-adjoint update with the alternating integration scheme used by GenPO while keeping the remaining components fixed. The resulting comparison in Figure 8 shows that the self-adjoint Verlet-style update provides consistent gains over the alternating scheme, supporting the role of time-symmetric integration in likelihood-based generative policy optimization.

(3) Likelihood stability during training. Appendix C.3 compares the standard deviation of policy likelihood ratios during training for GenPO and SAFLOW. SAFLOW exhibits smaller fluctuations as shown in Figure 9, suggesting that its self-adjoint discretization yields more stable and accurate forward-backward likelihood evaluation during policy updates.

281 **(4) Toy validation of discretization and likelihood accuracy.** Finally, Appendix C.4 evaluates
 282 both methods on a toy problem with an analytic solution. The results verify the expected second-
 283 order convergence of SAFLOW in both state discretization error and likelihood estimation error,
 284 complementing the empirical reinforcement-learning results with a controlled numerical validation.

285 5 Related Work

286 **Expressive Generative Policies.** Gaussian policies are widely used in policy-gradient algorithms
 287 because they provide tractable densities, efficient sampling, and stable likelihood-ratio estimation [2,
 288 1]. However, their limited expressiveness can be restrictive in high-dimensional continuous-control
 289 tasks, especially when effective behavior requires multimodal exploration, contact-rich interaction, or
 290 coordinated whole-body motion [8, 18]. This limitation has motivated more expressive stochastic
 291 policy classes, including normalizing-flow policies, implicit push-forward policies, energy-based
 292 policies, and diffusion- or flow-based policies [19, 20, 21, 22, 23]. Most of these methods are
 293 developed in off-policy settings, where policy improvement is driven by learned value functions [18],
 294 entropy-regularized objectives [24], or value-weighted generative training [22], rather than exact
 295 on-policy likelihood ratios. While these works demonstrate the benefits of expressive generative
 296 actors, they do not directly address the likelihood-tractability and numerical-consistency requirements
 297 of likelihood-based on-policy optimization.

298 **Likelihood-Based Generative Policy Optimization.** On-policy methods such as TRPO and PPO
 299 rely on accurate action likelihoods to compute importance ratios and enforce trust-region-style
 300 updates [2, 1]. This creates a central challenge for generative policies: expressive diffusion or
 301 flow-based samplers do not automatically provide stable and differentiable likelihood evaluation
 302 [6]. Recent work addresses this challenge either by avoiding explicit likelihoods through surrogate
 303 ratios [6, 9, 25], or by constructing invertible generative updates in augmented action spaces [11].
 304 These approaches narrow the gap between generative policies and likelihood-based RL, but exact
 305 invertibility alone does not ensure numerical consistency. In particular, if forward sampling and
 306 backward likelihood evaluation are based on asymmetric discretizations, the resulting likelihood
 307 ratios may inherit discretization artifacts. SAFLOW addresses this issue by introducing a self-adjoint
 308 flow in a doubled action space. Its update is inverted by applying the same rule with the opposite
 309 stepsize, aligning sampling and likelihood evaluation under a time-symmetric numerical scheme
 310 while providing a second-order approximation to the learned continuous flow.

311 6 Conclusion, Limitations and Future Work

312 We presented SAFLOW, a likelihood-tractable self-adjoint generative policy framework for on-policy
 313 reinforcement learning. SAFLOW defines an invertible flow in a doubled action space and uses
 314 a Verlet-style self-adjoint composition to make forward action sampling and backward likelihood
 315 evaluation consistent under stepsize reversal. This structure preserves exact numerical inversion
 316 while providing a second-order approximation to the underlying learned continuous flow. Across
 317 eight IsaacLab continuous-control tasks, SAFLOW consistently improves over Gaussian-policy
 318 baselines and recent generative-policy methods, highlighting the importance of numerically consistent
 319 likelihood evaluation for on-policy generative policy optimization.

320 We acknowledge that several limitations remain. First, SAFLOW uses a fixed number of flow steps
 321 across tasks. While this choice simplifies implementation and evaluation, different environments
 322 may benefit from different accuracy–efficiency trade-offs. Adaptive step allocation or early-stopping
 323 criteria for flow evaluation may further improve computational efficiency. Second, the compression
 324 coefficient is treated as a fixed hyperparameter. Although compression helps regularize redundant
 325 directions in the doubled action space, an adaptive schedule could reduce task-specific tuning and
 326 better balance expressiveness with stability during training. Third, the current evaluation focuses on
 327 state-based simulated continuous-control benchmarks. Extending SAFLOW to dexterous manipulation,
 328 vision-based control, and real-world robotic systems would provide a broader assessment of its
 329 scalability and robustness. Future work will investigate adaptive numerical schemes for generative
 330 policy evaluation, automatic regularization schedules for augmented action spaces, and applications
 331 of self-adjoint likelihood-tractable policies to more complex settings.

References

- [1] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [2] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015.
- [3] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.
- [4] Guojian Zhan, Letian Tao, Pengcheng Wang, Yixiao Wang, Yuxin Chen, Yiheng Li, Hongyang Li, Masayoshi Tomizuka, and Shengbo Eben Li. Mean flow policy with instantaneous velocity constraint for one-step action generation. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [5] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [6] Allen Z Ren, Justin Lidard, Lars L Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588*, 2024.
- [7] Michael Janner, Yilun Du, Joshua B Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022.
- [8] Zhendong Wang, Jonathan J Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. *arXiv preprint arXiv:2208.06193*, 2022.
- [9] David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, Hongsuk Choi, Haiwen Feng, and Angjoo Kanazawa. Flow matching policy gradients. *arXiv preprint arXiv:2507.21053*, 2025.
- [10] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [11] Shutong Ding, Ke Hu, Shan Zhong, Haoyang Luo, Weinan Zhang, Jingya Wang, Jun Wang, and Ye Shi. Genpo: Generative diffusion models meet on-policy reinforcement learning. *arXiv preprint arXiv:2505.18763*, 2025.
- [12] Christian Lubich and Gerhard Wanner. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*. Springer, 2006.
- [13] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- [14] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [15] George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.
- [16] Alexander Van-Brunt and Matt Visser. Explicit baker–campbell–hausdorff expansions. *Mathematics*, 6(8):135, 2018.
- [17] Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, Antonio Serrano-Munoz, Xinjie Yao, René Zurbügg, Nikita Rudin, et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.

- 377 [18] Yinuo Wang, Likun Wang, Mining Tan, Wenjun Zou, Xujie Song, Wenxuan Wang, Tong Liu,
378 Guojian Zhan, Tianze Zhu, Shiqi Liu, et al. Enhanced dacer algorithm with high diffusion
379 efficiency. *arXiv preprint arXiv:2505.23426*, 2025.
- 380 [19] Bogdan Mazouze, Thang Doan, Audrey Durand, Joelle Pineau, and R Devon Hjelm. Leveraging
381 exploration in off-policy algorithms via normalizing flows. In *Conference on Robot Learning*,
382 pages 430–444. PMLR, 2020.
- 383 [20] Lingwei Peng, Hui Qian, Zhebang Shen, Chao Zhang, and Fei Li. Generative actor-critic: An
384 off-policy algorithm using the push-forward model. *arXiv preprint arXiv:2105.03733*, 2021.
- 385 [21] Chen-Hao Chao, Chien Feng, Wei-Fang Sun, Cheng-Kuang Lee, Simon See, and Chun-Yi Lee.
386 Maximum entropy reinforcement learning via energy-based normalizing flow. *Advances in*
387 *Neural Information Processing Systems*, 37:56136–56165, 2024.
- 388 [22] Haitong Ma, Tianyi Chen, Kai Wang, Na Li, and Bo Dai. Efficient online reinforcement learning
389 for diffusion policy. *arXiv preprint arXiv:2502.00361*, 2025.
- 390 [23] Lunjun Zhang, Shuo Han, Hanrui Lyu, and Bradly C Stadie. D2 actor critic: Diffusion actor
391 meets distributional critic. *arXiv preprint arXiv:2510.03508*, 2025.
- 392 [24] Onur Celik, Zechu Li, Denis Blessing, Ge Li, Daniel Palenicek, Jan Peters, Georgia Chalvatzaki,
393 and Gerhard Neumann. Dime: Diffusion-based maximum entropy reinforcement learning.
394 *arXiv preprint arXiv:2502.02316*, 2025.
- 395 [25] Zechu Li, Rickmer Krohn, Tao Chen, Anurag Ajay, Pulkit Agrawal, and Georgia Chalvatzaki.
396 Learning multimodal behaviors from scratch with diffusion policy gradient. *Advances in Neural*
397 *Information Processing Systems*, 37:38456–38479, 2024.
- 398 [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez,
399 Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information*
400 *processing systems*, 30, 2017.

A Theoretical Analysis

This appendix provides full proofs for the theoretical results used in the main text. We first prove that SAFLOW’s doubled action space defines a valid augmented MDP whose optimal value matches that of the original MDP. We then prove that dummy-action likelihood ratios are valid policy-gradient ratios in the augmented MDP. Then, we prove numerical consistency results for the self-adjoint flow, including exact inversion, second-order likelihood consistency, and second-order perturbation bounds for policy ratios and PPO surrogates.

A.1 Optimality Preservation under Augmented Actions

Proof. We prove that the augmented MDP and the original MDP have the same optimal value, $\sup_{\tilde{\pi}} J_{\tilde{\mathcal{M}}}(\tilde{\pi}) = \sup_{\pi} J_{\mathcal{M}}(\pi)$, by establishing the two inequalities separately:

First, fix any augmented policy $\tilde{\pi}$. Define the original policy $\pi = M_{\#}\tilde{\pi}$ by

$$\pi(B|s) = \tilde{\pi}(M^{-1}(B)|s)$$

for every measurable set $B \subseteq \mathcal{A}$. Under $\tilde{\pi}$ in the augmented MDP, the environment receives the projected action $a = M(\tilde{a})$. Since

$$\tilde{P}(s'|s, \tilde{a}) = P(s'|s, M(\tilde{a}))$$

and

$$\tilde{r}(s, \tilde{a}) = r(s, M(\tilde{a})),$$

the joint distribution over state trajectories and projected actions under $\tilde{\pi}$ in $\tilde{\mathcal{M}}$ is identical to the joint distribution over state-action trajectories under π in $\mathcal{M}$. Therefore,

$$J_{\tilde{\mathcal{M}}}(\tilde{\pi}) = J_{\mathcal{M}}(M_{\#}\tilde{\pi}) \leq \sup_{\pi} J_{\mathcal{M}}(\pi).$$

Taking the supremum over $\tilde{\pi}$ gives

$$\sup_{\tilde{\pi}} J_{\tilde{\mathcal{M}}}(\tilde{\pi}) \leq \sup_{\pi} J_{\mathcal{M}}(\pi).$$

Second, fix any original policy π . Define its diagonal lift $\tilde{\pi}^{\Delta}$ by sampling $a \sim \pi(\cdot|s)$ and setting

$$\tilde{a} = (x, y) = (a, a).$$

Equivalently,

$$\tilde{\pi}^{\Delta}(dx, dy|s) = \int_{\mathcal{A}} \delta_a(dx) \delta_a(dy) \pi(da|s).$$

For this lifted policy,

$$M(x, y) = a$$

almost surely. Hence the state-action trajectory in $\tilde{\mathcal{M}}$ under $\tilde{\pi}^{\Delta}$ coincides with the trajectory in $\mathcal{M}$ under π , and

$$J_{\tilde{\mathcal{M}}}(\tilde{\pi}^{\Delta}) = J_{\mathcal{M}}(\pi).$$

Taking the supremum over π gives

$$\sup_{\tilde{\pi}} J_{\tilde{\mathcal{M}}}(\tilde{\pi}) \geq \sup_{\pi} J_{\mathcal{M}}(\pi).$$

Combining both inequalities proves the claim. $\square$

A.2 Policy Gradient in the Augmented Action Space

Proof. We apply the standard policy-gradient theorem to the augmented MDP

$$\tilde{\mathcal{M}} = (\mathcal{S}, \tilde{\mathcal{A}}, \tilde{P}, \tilde{r}, \rho_0, \gamma).$$

In this MDP, the action variable is the dummy action $\tilde{a}$, and the policy density is $\tilde{\pi}_{\theta}(\tilde{a}|s)$. Hence, the score function appearing in the policy gradient is

$$\nabla_{\theta} \log \tilde{\pi}_{\theta}(\tilde{a}_t|s_t).$$

429 The reward in the augmented MDP is induced by the projected environment action:

$$\tilde{r}(s_t, \tilde{a}_t) = r(s_t, M(\tilde{a}_t)).$$

430 Therefore, the corresponding advantage $A^{\tilde{\pi}_\theta}(s_t, \tilde{a}_t)$ is the advantage of taking the dummy action $\tilde{a}_t$
 431 in the augmented MDP, with its effect on the environment determined by $M(\tilde{a}_t)$. The policy-gradient
 432 theorem then gives

$$\nabla_\theta J_{\tilde{\mathcal{M}}}(\theta) = \mathbb{E}_{(s, \tilde{a}) \sim d^{\tilde{\pi}_\theta}} \left[\nabla_\theta \log \tilde{\pi}_\theta(\tilde{a}|s) A^{\tilde{\pi}_\theta}(s, \tilde{a}) \right],$$

433 where $d^{\tilde{\pi}_\theta}$ denotes the discounted state–action occupancy measure in $\tilde{\mathcal{M}}$.

434 When rollout samples are collected from $\tilde{\pi}_{\theta_{\text{old}}}$ and reused to update $\tilde{\pi}_\theta$, the change of measure is also
 435 performed in the augmented action space. Thus, the corresponding one-step importance ratio is

$$r_\theta(s, \tilde{a}) = \frac{\tilde{\pi}_\theta(\tilde{a}|s)}{\tilde{\pi}_{\theta_{\text{old}}}(\tilde{a}|s)},$$

436 which is exactly the likelihood ratio used by SAFLOW for PPO-style optimization. $\square$

437 A.3 Exact Inversion of the Self-Adjoint Flow

438 *Proof.* Let $z = (x, y)$ denote the dummy action. SAFLOW uses a symmetric split update of the form

$$\Psi_h^\theta = \phi_x^{h/2} \circ \phi_y^h \circ \phi_x^{h/2},$$

439 where ϕ_x^h and ϕ_y^h are the two sub-updates associated with the split dynamics. The K -step map is

$$F_{\theta, h} = (\Psi_h^\theta)^K, \quad T = Kh = 1.$$

440 Each split update is inverted by changing the sign of its stepsize:

$$(\phi_x^{h/2})^{-1} = \phi_x^{-h/2}, \quad (\phi_y^h)^{-1} = \phi_y^{-h}.$$

441 Using the inverse rule for compositions,

$$(A \circ B \circ C)^{-1} = C^{-1} \circ B^{-1} \circ A^{-1},$$

442 we obtain

$$\begin{aligned} (\Psi_h^\theta)^{-1} &= \left(\phi_x^{h/2} \circ \phi_y^h \circ \phi_x^{h/2} \right)^{-1} \\ &= \phi_x^{-h/2} \circ \phi_y^{-h} \circ \phi_x^{-h/2} \\ &= \Psi_{-h}^\theta. \end{aligned}$$

443 Since

$$F_{\theta, h} = \underbrace{\Psi_h^\theta \circ \dots \circ \Psi_h^\theta}_{K \text{ times}},$$

444 its inverse is obtained by composing the inverse one-step map K times:

$$F_{\theta, h}^{-1} = \underbrace{\Psi_{-h}^\theta \circ \dots \circ \Psi_{-h}^\theta}_{K \text{ times}} = (\Psi_{-h}^\theta)^K.$$

445 $\square$

446 A.4 Second-Order Approximation of the Flow

447 *Proof.* The SAFLOW one-step update is the symmetric Strang-type composition

$$\Psi_h^\theta = \phi_x^{h/2} \circ \phi_y^h \circ \phi_x^{h/2}.$$

448 For sufficiently smooth vector fields, the Baker–Campbell–Hausdorff expansion of symmetric splitting
 449 methods shows that even-order local error terms cancel. As a result, the local truncation error of one
 450 step is

$$\Psi_h^\theta(z) - \varphi_h^\theta(z) = \mathcal{O}(h^3),$$

uniformly on compact sets. Over a fixed horizon $T = Kh$, the number of steps is $K = T/h$, so accumulation of the local truncation error gives the global state error

$$\sup_{z_0} \|F_{\theta,h}(z_0) - \varphi_T^\theta(z_0)\| = K\mathcal{O}(h^3) = \mathcal{O}(h^2).$$

For the C^1 estimate, consider the variational equation of the exact flow and the corresponding linearized numerical flow. Under one additional derivative of smoothness, the same symmetric splitting argument applies to the variational dynamics. Thus the local Jacobian error is $\mathcal{O}(h^3)$, and accumulation over $K = T/h$ steps gives

$$\sup_{z_0} \|DF_{\theta,h}(z_0) - D\varphi_T^\theta(z_0)\| = \mathcal{O}(h^2).$$

Combining the state and Jacobian estimates yields the C^1 bound. $\square$

A.5 Second-Order Likelihood Consistency

Proof. Let ρ_0 be the base density over the initial dummy action z_0 . The discretized SAFLOW density is computed by change of variables:

$$\ell_{\theta,h}(s, z) = \log \tilde{\pi}_{\theta,h}(z|s) = \log p_0(F_{\theta,h}^{-1}(z)) + \log \left| \det DF_{\theta,h}^{-1}(z) \right|.$$

Similarly, the exact continuous-flow likelihood is

$$\ell_{\theta,\text{cont}}(s, z) = \log \rho_0((\varphi_T^\theta)^{-1}(z)) + \log \left| \det D(\varphi_T^\theta)^{-1}(z) \right|.$$

Because φ_T^θ is a diffeomorphism with Jacobian determinant bounded away from zero, the inverse map is locally stable under C^1 perturbations. Since

$$\|F_{\theta,h} - \varphi_T^\theta\|_{C^1} = \mathcal{O}(h^2),$$

we have

$$\|F_{\theta,h}^{-1} - (\varphi_T^\theta)^{-1}\|_{C^1} = \mathcal{O}(h^2)$$

on the generated compact domain.

We decompose the log-likelihood error into the base-density term and the log-determinant term:

$$\begin{aligned} & |\ell_{\theta,h}(s, z) - \ell_{\theta,\text{cont}}(s, z)| \\ & \leq \left| \log p_0(F_{\theta,h}^{-1}(z)) - \log \rho_0((\varphi_T^\theta)^{-1}(z)) \right| \\ & \quad + \left| \log \left| \det DF_{\theta,h}^{-1}(z) \right| - \log \left| \det D(\varphi_T^\theta)^{-1}(z) \right| \right|. \end{aligned}$$

For the first term, since ρ_0 is positive and continuously differentiable on a compact set, $\log \rho_0$ is Lipschitz on that set. Therefore,

$$\left| \log p_0(F_{\theta,h}^{-1}(z)) - \log \rho_0((\varphi_T^\theta)^{-1}(z)) \right| = \mathcal{O}(h^2).$$

For the second term, the C^1 inverse closeness gives

$$DF_{\theta,h}^{-1}(z) = D(\varphi_T^\theta)^{-1}(z) + \mathcal{O}(h^2).$$

Because the determinant of $D(\varphi_T^\theta)^{-1}(z)$ is bounded away from zero, the map

$$A \mapsto \log |\det A|$$

is Lipschitz in a neighborhood of this Jacobian matrix. Hence

$$\left| \log \left| \det DF_{\theta,h}^{-1}(z) \right| - \log \left| \det D(\varphi_T^\theta)^{-1}(z) \right| \right| = \mathcal{O}(h^2).$$

Combining the two estimates proves the result. $\square$

473 **A.6 Policy-Ratio and PPO-Surrogate Consistency**

474 *Proof.* By Theorem 5 and its proof in Appendix A.5,

$$\ell_{\theta,h}(s, z) = \ell_{\theta,\text{cont}}(s, z) + \mathcal{O}(h^2),$$

475 and

$$\ell_{\theta_{\text{old}},h}(s, z) = \ell_{\theta_{\text{old}},\text{cont}}(s, z) + \mathcal{O}(h^2).$$

476 Subtracting the two identities gives

$$\ell_{\theta,h}(s, z) - \ell_{\theta_{\text{old}},h}(s, z) = \ell_{\theta,\text{cont}}(s, z) - \ell_{\theta_{\text{old}},\text{cont}}(s, z) + \mathcal{O}(h^2).$$

477 Exponentiating both sides yields

$$r_{\theta,h}(s, z) = r_{\theta,\text{cont}}(s, z) \exp(\mathcal{O}(h^2)).$$

478 If $r_{\theta,\text{cont}}$ is bounded, then

$$\exp(\mathcal{O}(h^2)) - 1 = \mathcal{O}(h^2),$$

479 which implies the additive bound.

480 For a fixed advantage value A , define

$$g_A(r) = \min(rA, \text{clip}(r, 1 - \epsilon, 1 + \epsilon)A).$$

481 The map $r \mapsto rA$ is $|A|$ -Lipschitz. The clipped map $r \mapsto \text{clip}(r, 1 - \epsilon, 1 + \epsilon)A$ is also $|A|$ -Lipschitz.

482 The pointwise minimum of two functions with Lipschitz constant $|A|$ is also $|A|$ -Lipschitz. Therefore,

$$|g_A(r) - g_A(r')| \leq |A||r - r'|.$$

483 Using $A = \hat{A}_t$, $r = r_{\theta,h}$, and $r' = r_{\theta,\text{cont}}$, we get

$$|g_{\hat{A}_t}(r_{\theta,h}) - g_{\hat{A}_t}(r_{\theta,\text{cont}})| \leq |\hat{A}_t||r_{\theta,h} - r_{\theta,\text{cont}}|.$$

484 By the bounded-advantage assumption and policy-ratio consistency,

$$|g_{\hat{A}_t}(r_{\theta,h}) - g_{\hat{A}_t}(r_{\theta,\text{cont}})| = \mathcal{O}(h^2).$$

485 Taking expectation over rollout samples preserves the order, yielding

$$|L_{\text{clip},h}(\theta) - L_{\text{clip},\text{cont}}(\theta)| = \mathcal{O}(h^2).$$

486

□

B Environmental Details

B.1 Benchmark Introduction

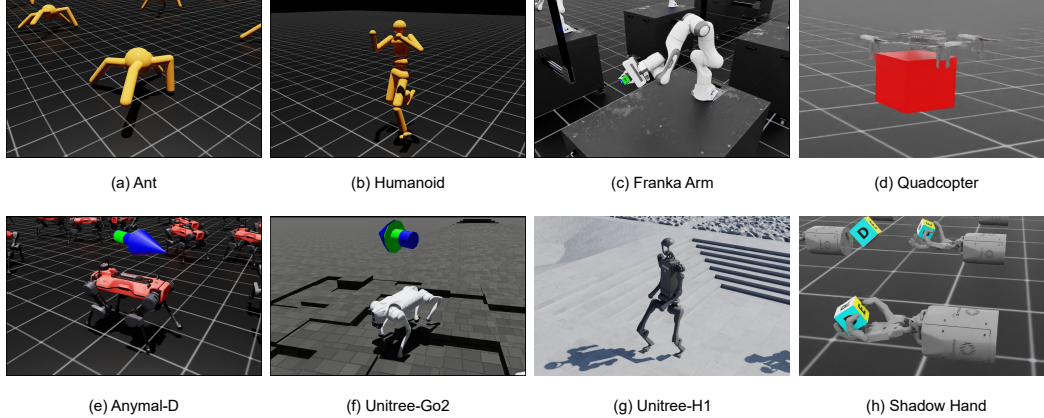


Figure 5: Eight IsaacLab benchmark visualizations.

IsaacLab¹ serves as the experimental platform in this work. It is a robot-learning framework built upon NVIDIA Isaac Sim and is designed to support scalable research on reinforcement learning, imitation learning, and planning. By integrating the GPU-accelerated PhysX simulator and efficient rendering interfaces, IsaacLab enables parallelized simulation for a wide range of robotic systems and control scenarios.

Table 2: Summary of the IsaacLab benchmark environments used in our experiments.

Environment	Category	State Dim.	Action Dim.
Isaac-Ant-v0	Classic control	60	8
Isaac-Humanoid-v0	Classic control	87	21
Isaac-Lift-Cube-Franka-v0	Arm manipulation	36	8
Isaac-Repose-Cube-Shadow-Direct-v0	Dexterous manipulation	157	20
Isaac-Velocity-Flat-Anymal-D-v0	Quadruped locomotion	48	12
Isaac-Velocity-Rough-Unitree-Go2-v0	Rough-terrain locomotion	235	12
Isaac-Velocity-Rough-H1-v0	Humanoid locomotion	256	19
Isaac-Quadcopter-Direct-v0	Aerial control	12	4

The selected environments are summarized in Table 2. The classical control tasks, *Isaac-Ant-v0* and *Isaac-Humanoid-v0*, evaluate basic locomotion capability in continuous action spaces. The manipulation tasks include *Isaac-Lift-Cube-Franka-v0*, where a Franka arm must lift and move a cube to a target location, and *Isaac-Repose-Cube-Shadow-Direct-v0*, where a Shadow Dexterous Hand performs in-hand cube reorientation. For legged locomotion, we include *Isaac-Velocity-Flat-Anymal-D-v0*, *Isaac-Velocity-Rough-Unitree-Go2-v0*, and *Isaac-Velocity-Rough-H1-v0*, which require robots with different morphologies to track commanded velocities under flat or rough-terrain conditions. Finally, *Isaac-Quadcopter-Direct-v0* evaluates aerial control by requiring a quadcopter to reach and hover around a target position through thrust commands.

Overall, this benchmark suite covers a broad spectrum of control difficulties, from low-dimensional aerial stabilization to high-dimensional humanoid and dexterous-hand control. The large variation in observation size, action dimension, robot morphology, and contact complexity makes it suitable for testing both the performance and robustness of policy optimization algorithms.

¹<https://isaac-sim.github.io/IsaacLab/main/index.html>

Table 3: Hyper-parameter settings used in the experiments.

(a) Environment-specific hyper-parameters.				
Task	Max Iteration	Steps per Env.	Policy/Value Size	Policy/Value LR
Ant	1000	32	[400, 200, 100]	0.0010
Humanoid	2000	32	[400, 200, 100]	0.0005
Franka Arm	1500	24	[256, 128, 64]	0.0001
Quadcopter	500	24	[64, 64]	0.0005
Shadow Hand	10000	16	[512, 512, 256, 128]	0.0005
Anymal-D	300	24	[128, 128, 128]	0.0010
Unitree-H1	3000	24	[512, 256, 128]	0.0010
Unitree-Go2	1500	24	[512, 256, 128]	0.0010

(b) Shared hyper-parameters across all environments.			
Parameter	Value	Parameter	Value
<i>Training Configuration</i>			
Number of environments	2048	Number of learning epochs	5
Mini-batch size	4	Gradient norm clipping	1.0
<i>RL and Optimization Parameters</i>			
Policy/value activation	Mish	Policy ratio clipping	0.2
Time embedding dimension	32	Hidden layers in time embedding	[256, 256]
Discount factor	0.99	GAE coefficient λ	0.95
Compression coefficient	0.01	Flow steps	5

507 B.2 SAFLOW Model Architecture

508 The actor network, corresponding to the flow policy, is implemented as a multi-layer perceptron
 509 (MLP) with hidden layer dimensions. The input to the actor consists of the concatenation of the
 510 current state, action, and sinusoidal time embedding vectors, resulting in an input dimension of
 511 $|\mathcal{A}| + |\mathcal{S}| + |\mathcal{T}|$. The output dimension is equal to the action dimension $|\mathcal{A}|$, representing the flow-
 512 based action refinement. During training, instead of directly concatenating the time step t , state s_t ,
 513 and action a_t as raw input features, we incorporate sinusoidal positional embeddings to encode the
 514 temporal component [26]. This approach, inspired by the positional encoding technique, maps each
 515 timestep $t \in \{0, 1, \dots, T\}$ to a fixed-dimensional vector $e_t \in \mathbb{R}^d$ as follows:

$$e_t^{(2i)} = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad e_t^{(2i+1)} = \cos\left(\frac{t}{10000^{2i/d}}\right), \quad \text{for } i = 0, 1, \dots, \frac{d}{2} - 1. \quad (8)$$

516 The resulting embedding e_t is then passed through a MLP to allow for nonlinear transformation and
 517 projection into the model’s feature space. The MLP output is subsequently concatenated with the
 518 state s_t and action a_t to form the final input representation:

$$\tilde{x}_t = \text{concat}(s_t, a_t, \text{Embed}(e_t)), \quad (9)$$

519 which enriches the model input with a smooth, continuous representation of temporal progression.
 520 Unlike raw scalar timestep concatenation, sinusoidal embeddings provide a structured encoding that
 521 helps the model infer temporal relationships and ordering, even in the absence of explicit recurrence
 522 or attention mechanisms. The critic network is also implemented as an MLP with the same hidden
 523 layer structure. It receives only the current state as input and outputs a scalar value corresponding to
 524 the estimated state value.

525 B.3 Reproducibility Statement & Detailed Hyperparameters

526 Our proposed method, SAFLOW, is implemented using RSL-RL, a lightweight and high-performance
 527 framework tailored for robotics and continuous control, which emphasizes computational efficiency
 528 and ease of deployment. For each baseline algorithm, we referred to its official core implemen-
 529 tation and integrated it into the same codebase. For classical PPO and TRPO, we use the official

implementation in <https://github.com/isaac-sim/IsaacLab>. For FPO, we use the official implementation in <https://github.com/akanazawa/fpo>. For GenPO, we use the official implementation in <https://github.com/wadx2019/genpo>. The detailed hyper-parameters are listed in Table 3.

All experiments were conducted on a workstation equipped with an Intel Core i9-12900K CPU, 128 GB of system memory, and two NVIDIA A100 GPUs, each with 80 GB of GPU memory. For the software environment, we used Isaac Sim 4.5.0 together with IsaacLab 0.36.3. The benchmark environments were built from isaacsim_tasks 0.10.27, isaacsim_r1 0.1.3, and isaacsim_assets 0.2.1. GPU acceleration relied on CUDA 12.4-related NVIDIA libraries, with cuDNN 9.1.0 and NCCL 2.21.5. All benchmark experiments, including training and evaluation, were performed under the same configuration to ensure a fair comparison across different methods.

C Additional Experiments

C.1 Time Efficiency Analysis.

We further report the wall-clock training time on the Ant and Humanoid environments to evaluate the computational efficiency of different algorithms. Each method is evaluated with 5 random seeds, and we report the mean and standard deviation of the elapsed time.

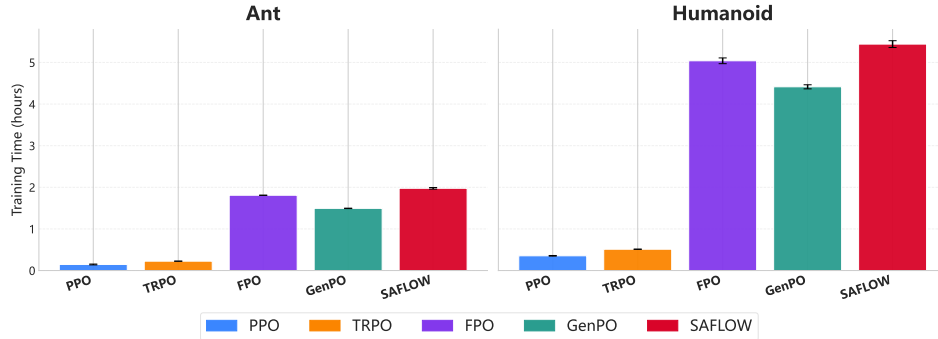


Figure 6: Comparison of training time efficiency.

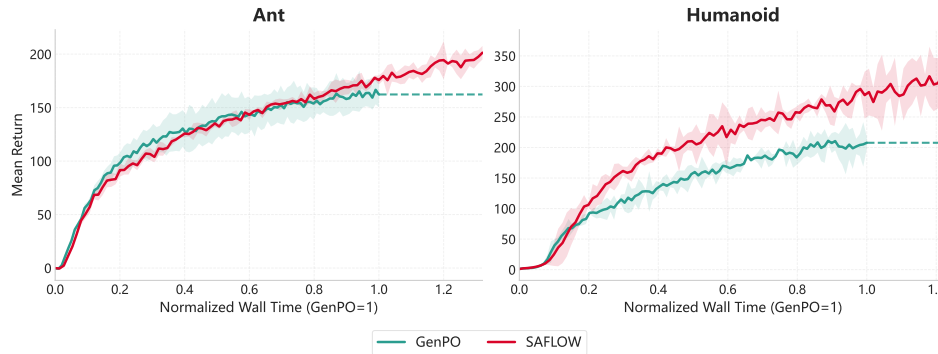


Figure 7: Curves of Performance vs wall time.

As shown in Figures 6 and 7 and Table 4, PPO and TRPO require the least wall-clock time, which is expected since they rely on relatively simple Gaussian policy parameterizations. In contrast, generative-policy methods introduce additional computation due to iterative action generation and more expressive policy modeling. Among these methods, SAFLOW requires more training time than FPO and GenPO on both Ant and Humanoid. However, considering the substantial performance improvements reported in the main results, this additional computational cost is acceptable when final policy performance is the primary concern, particularly for challenging control tasks where higher returns are desired.

Table 4: Wall-clock training time on the Ant and Humanoid environments.

Algorithm	Ant		Humanoid	
	Time (s)	Time (h)	Time (s)	Time (h)
PPO	527.33 ± 34.77	0.146 ± 0.010	1275.77 ± 28.40	0.354 ± 0.008
TRPO	809.04 ± 4.54	0.225 ± 0.001	1844.24 ± 15.61	0.512 ± 0.004
FPO	6511.85 ± 11.33	1.809 ± 0.003	18142.87 ± 248.16	5.040 ± 0.069
GenPO	5379.82 ± 18.52	1.494 ± 0.005	15893.10 ± 178.19	4.415 ± 0.049
SAFLOW	7115.62 ± 69.90	1.976 ± 0.019	19589.14 ± 296.93	5.441 ± 0.082

554 C.2 Controlled Comparison of Flow Policy Integration Schemes.

555 To isolate the effect of the integration scheme, we compare the default second-order self-adjoint
 556 update with a first-order alternating variant implemented in the same SAFLOW codebase. The second-
 557 order variant consistently achieves higher returns, indicating that the performance gain majorly comes
 558 from the self-adjoint integration structure rather than from other implementation differences.

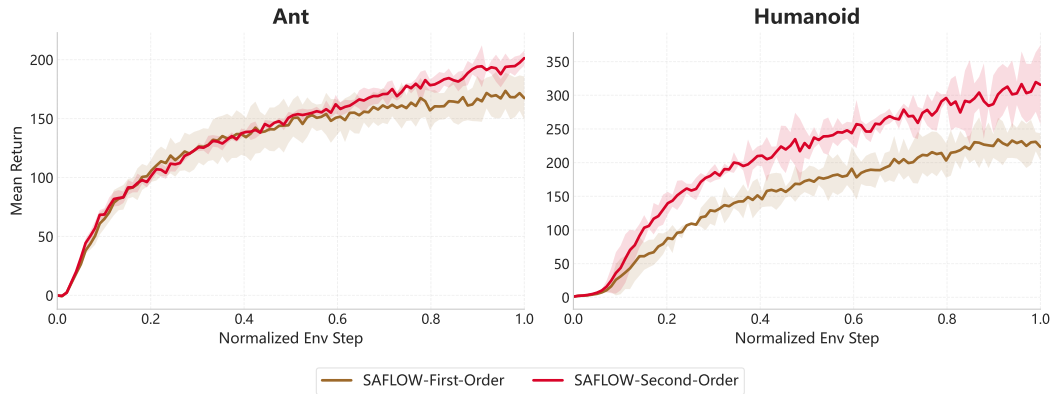


Figure 8: Controlled comparison of flow policy integration schemes.

559 C.3 Likelihood Stability During Training.

560 We recorded the standard deviation of the likelihood ratio for each batch during training. The
 561 results show that SAFLOW has a lower standard deviation, suggesting more stable and accurate
 562 likelihood-ratio estimates.

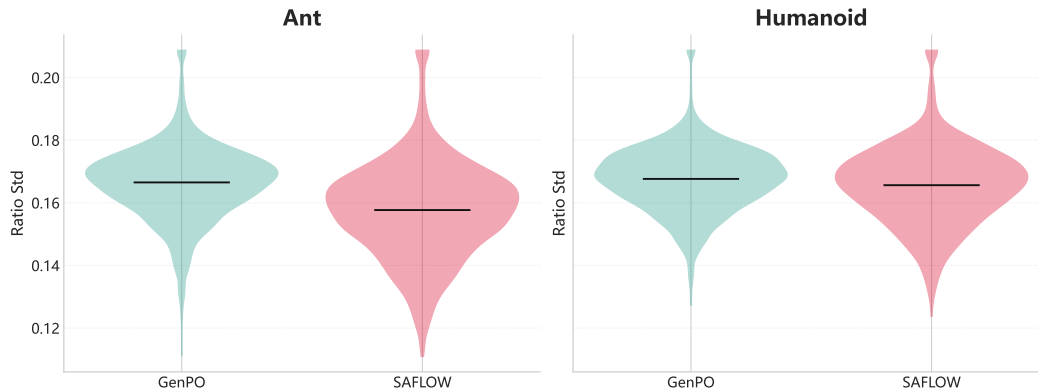


Figure 9: Likelihood stability comparison between GenPO and SAFLOW during training.

563 C.4 Toy Validation of Discretization and Likelihood Accuracy.

564 **Objective.** We evaluate whether SAFLOW yields higher discretization accuracy and more accurate
565 likelihood estimation than GenPO under a controlled setting with closed-form ground truth.

566 **Experimental protocol.** We consider the 2D time-dependent system

$$\frac{dx}{dt} = \omega(t)y, \quad \frac{dy}{dt} = \omega(t)x, \quad \omega(t) = 1 + 0.5 \sin(2\pi t), \quad t \in [0, 1]. \quad (10)$$

567 With $u = x + y$ and $v = x - y$, the system decouples into

$$\frac{du}{dt} = \omega(t)u, \quad \frac{dv}{dt} = -\omega(t)v, \quad (11)$$

568 which admits an analytic solution. Therefore, both trajectory error and likelihood error can be
569 measured against exact targets.

570 We sample $(x_0, y_0) \sim \mathcal{N}(0, I)$ with 10,000 samples, and sweep the number of integration steps

$$K \in \{2, 4, 8, 16, 32, 64, 128\}. \quad (12)$$

571 **Metrics.** For each K , we report:

$$\text{StateErr}(K) = \mathbb{E} [\|z_1^{\text{num}} - z_1^*\|_2], \quad (13)$$

$$\text{InvErr}(K) = \mathbb{E} [\|z_0^{\text{inv}} - z_0\|_2], \quad (14)$$

$$\text{LLErr}(K) = \mathbb{E} [|\log p_{\text{num}}(z_1^*) - \log p_*(z_1^*)|]. \quad (15)$$

572 We estimate empirical convergence order by

$$\text{order}(K \rightarrow 2K) = \log_2 \frac{\text{err}(K)}{\text{err}(2K)}. \quad (16)$$

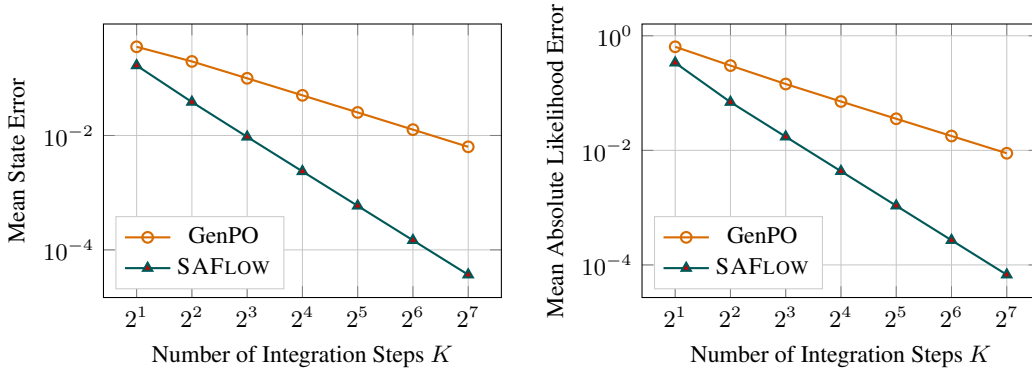


Figure 10: Log-log error curves versus integration steps K . Left: mean state error. Right: mean absolute likelihood error. SAFLOW exhibits a steeper slope, indicating higher-order convergence.

573 **Results and analysis.** Figure 10 shows that both methods exhibit approximately linear trends on
574 log-log axes, while SAFLOW has a visibly steeper slope for both state error (left) and likelihood error
575 (right). This indicates a faster error decay rate as K increases. Table 5 confirms the same pattern
576 numerically across all tested K .

577 For concrete comparisons from Table 5, at $K = 8$, SAFLOW reduces state error from 9.999×10^{-2}
578 to 9.456×10^{-3} (about $10.6\times$ lower) and likelihood error from 1.438×10^{-1} to 1.728×10^{-2} (about
579 $8.3\times$ lower). At $K = 32$, the gains are larger: state error is reduced by about $43\times$ (2.529×10^{-2} vs.
580 5.882×10^{-4}), inverse error by about $31\times$ (4.014×10^{-2} vs. 1.289×10^{-3}), and likelihood error
581 by about $33\times$ (3.558×10^{-2} vs. 1.079×10^{-3}).

582 Table 6 summarizes the estimated convergence order and matches the visual trend in Fig. 10: GenPO is
583 close to first-order (state: 0.9920, likelihood: 1.0168), while SAFLOW is close to second-order (state:
584 2.0062, likelihood: 2.0030). Overall, at matched computational discretization level K , SAFLOW
585 consistently provides higher discretization fidelity and more accurate likelihood estimation.

Table 5: Numerical errors for GenPO and SAFLOW under different numbers of integration steps K .

K	State Error		Inverse Error		Likelihood Error	
	GenPO	SAFLOW	GenPO	SAFLOW	GenPO	SAFLOW
2	0.355545	0.167358	0.626692	0.379097	0.642042	0.337652
4	0.197274	0.038445	0.327340	0.082160	0.301857	0.069460
8	0.099987	0.009455	0.160986	0.020607	0.143792	0.017278
16	0.050374	0.002355	0.080285	0.005154	0.071244	0.004316
32	0.025285	0.000588	0.040140	0.001288	0.035578	0.001078
64	0.012667	0.000147	0.020074	0.000322	0.017794	0.000269
128	0.006339	0.000036	0.010039	0.000080	0.008900	0.000067

Table 6: Empirical convergence order (mean over $K \geq 4$).

Error Order	GenPO	SAFLOW
State error	0.9920	2.0062
Likelihood error	1.0168	2.0030

586 C.5 Complete Learning Curves for the Scaling Experiment.

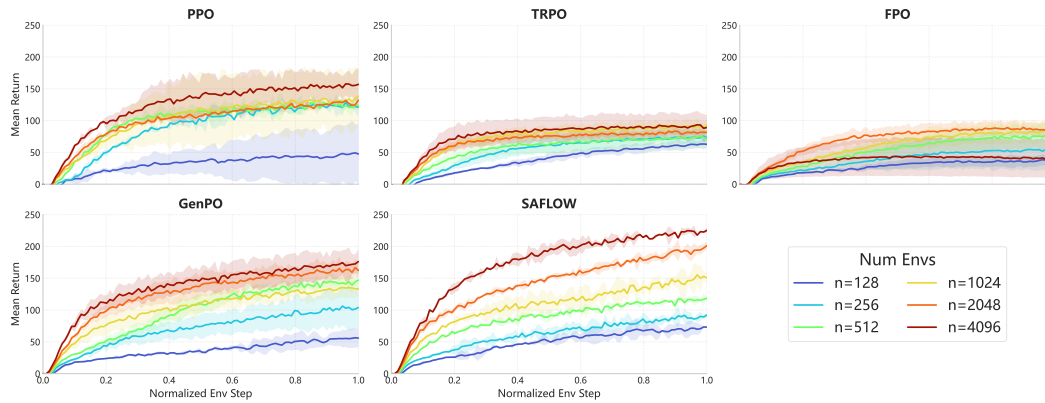


Figure 11: Full Learning curves of scaling experiment.

587 D Societal Impacts

588 **Potential Positive Societal Impacts.** This work improves generative policy optimization for
589 continuous-control reinforcement learning by enabling more expressive and numerically consistent
590 policy parameterizations. In the long term, such methods may benefit robotics and embodied AI
591 systems that require robust control in complex environments, including assistive robotics, industrial
592 automation, logistics, and autonomous inspection. More stable and sample-efficient policy optimiza-
593 tion could reduce the computational cost of training RL agents, lowering energy consumption and
594 making advanced control methods more accessible to smaller research groups.

595 **Potential Negative Societal Impacts.** As with many advances in RL and robotics, this work may
596 also contribute to the development of autonomous systems that could be deployed in unsafe or
597 harmful ways if not properly constrained. More capable continuous control policies may improve
598 the performance of robots, drones, or other automated platforms in settings where failures, misuse,
599 or insufficient human oversight could pose physical risks. In addition, stronger RL methods may
600 increase reliance on simulation-trained policies whose real-world behavior is not fully verified.
601 Careful evaluation, safety constraints, human oversight, and domain-specific deployment standards
602 are therefore important when applying these methods outside controlled benchmarks.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: the main claims in the abstract and introduction accurately reflect the paper’s contributions and scope. Please refer to the Abstract and Section 1.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Limitations are discussed in Section 6.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Assumptions and complete proofs are provided in Appendix A.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Implementation details are described in Appendix B.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The training code is provided in the supplemental material. Detailed information regarding the benchmark and configuration is available in Appendix B.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Detailed information regarding the benchmark and configuration is available in Appendix B.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We run 5 random seeds for each experimental setting and report the std to reflect the statistical significance of the experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).

- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Detailed information regarding the compute resources is available in Appendix B.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: Our research adheres to the established ethical guidelines

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Both potential positive societal impacts and negative societal impacts can be found in Appendix B.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification:

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All the assets used in the paper are credited and explicitly mentioned and properly respected.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: No new assets in the paper.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification:

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification:

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

917 Question: Does the paper describe the usage of LLMs if it is an important, original, or
918 non-standard component of the core methods in this research? Note that if the LLM is used
919 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
920 scientific rigor, or originality of the research, declaration is not required.

921 Answer: [N/A]

922 Justification: The core method development in this research does not involve LLMs as any
923 important, original, or non-standard components.

924 Guidelines:

- 925 • The answer [N/A] means that the core method development in this research does not
926 involve LLMs as any important, original, or non-standard components.
- 927 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
928 be described.